

Agent Phantom Recovery

14-llms-integration.md

Production LLM Integration Architecture

Version: 1.0

Status: Production Design

1. Purpose

This document defines the Large Language Model (LLM) architecture for Agent Phantom Recovery.

The goal of this architecture is not simply to generate responses.

The goal is to enable:

- Long-horizon execution
- Repository understanding
- Multi-step reasoning
- Tool orchestration
- Autonomous investigation
- Root cause analysis
- Patch generation
- Verification workflows
- Continuous replanning

within the Antigravity IDE environment.

2. Design Philosophy

Traditional AI Architecture

Most AI applications follow:

```
User
↓
Single Model
↓
Response
```

This architecture is simple but creates limitations.

Problems:

- Single point of failure
- Shared weaknesses
- No independent verification
- Limited specialization

Agent Phantom Recovery Architecture

```
Planner
↓
Reasoner
↓
Verifier
```

Each model specializes in a different cognitive role.

This creates:

- Better reasoning quality
- Better verification quality
- Better reliability
- Better scalability

3. Selected Model Stack

Version 1 architecture:

```
Planner
↓
Kimi K2.6 Free

Reasoner
↓
Qwen3 Coder 480B Free

Verifier
↓
Nemotron 3 Ultra Free
```

Provider:

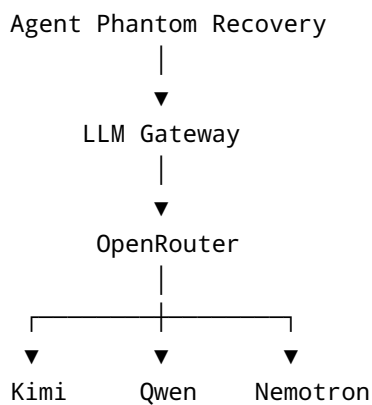
4. Why OpenRouter

OpenRouter acts as the model routing layer.

Benefits:

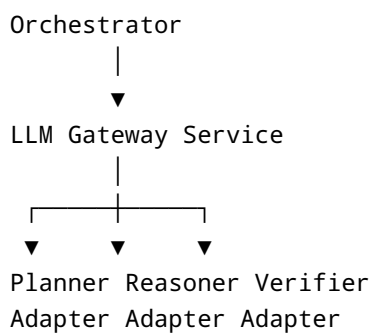
- Unified API
- Provider independence
- Failover flexibility
- Easy model replacement
- Lower operational complexity

Architecture



The backend never directly depends on a specific vendor API.

5. LLM Layer Architecture





This abstraction prevents vendor lock-in.

6. Core Principle

Models are not exposed directly to the rest of the platform.

Other services interact with:

Planner Service
Reasoner Service
Verifier Service

Never:

Gemini Service
Qwen Service
Nemotron Service

This separation protects the architecture from future model changes.

7. Planner Integration

Model

Kimi K2.6 Free

Role

Responsible for:

Goal Analysis
Task Decomposition
Execution Planning
Tool Selection

Replanning
Recovery Planning

Inputs

Planner receives:

Goal
Task State
Memory Summary
Tool Results Summary
Current Plan

Planner Should Not Receive

Avoid sending:

Entire Repository
Entire File Tree
Large Codebases

Planner should focus on strategy.

Outputs

Planner produces:

```
{  
  "plan": [],  
  "next_actions": [],  
  "success_criteria": []  
}
```

8. Reasoner Integration

Model

Qwen3 Coder 480B Free

Role

Responsible for:

Code Analysis
Repository Understanding
Root Cause Analysis
Patch Generation
Architecture Reasoning
Dependency Analysis

Inputs

Reasoner receives:

Relevant Files
Repository Chunks
Dependency Graph
Call Graph
Evidence
Planner Instructions

Reasoner Should Not Receive

Avoid:

Entire Project
Irrelevant Memory
Unused Files

Context should be highly curated.

Outputs

Reasoner produces:

```
{
  "findings": [],
  "patches": [],
  "confidence": 0.94
}
```

9. Verifier Integration

Model

Nemotron 3 Ultra Free

Role

Responsible for:

Verification
Patch Review
Regression Detection
Evidence Validation
Risk Analysis

Inputs

Verifier receives:

Finding
Evidence
Patch
Test Results
Reasoning Summary

Verifier Mission

Verifier should actively challenge conclusions.

Workflow:

```
Reasoner Output
↓
Critique
↓
Review
↓
Approve / Reject
```

Outputs

```
{
  "approved": true,
  "confidence": 0.97,
  "concerns": []
}
```

10. LLM Gateway Service

Purpose

Single entry point for all model communication.

Responsibilities

```
Routing
Authentication
Retries
Rate Limiting
Logging
Monitoring
Fallbacks
```


Architecture

```
Orchestrator
  ↓
LLM Gateway
  ↓
Model Adapters
  ↓
OpenRouter
```

11. Model Adapters

Each model has a dedicated adapter.

Planner Adapter

```
planner_adapter
```

Responsibilities:

- Prompt construction
- Response validation
- Parsing

Reasoner Adapter

```
reasoner_adapter
```

Responsibilities:

- Repository context assembly
- Patch extraction
- Structured outputs

Verifier Adapter

```
verifier_adapter
```

Responsibilities:

- Review prompts
 - Verification outputs
 - Approval decisions
-

12. Prompt Architecture

Principle

Prompts should be role-specific.

Planner Prompt

Focus:

Planning
Task Decomposition
Execution Strategy

Reasoner Prompt

Focus:

Code Understanding
Root Cause Analysis
Patch Design

Verifier Prompt

Focus:

Critique
Validation
Regression Detection

13. Structured Output Enforcement

All models must return structured outputs.

Planner

```
{
  "plan": []
}
```

Reasoner

```
{
  "findings": [],
  "patches": []
}
```

Verifier

```
{
  "approved": true,
  "concerns": []
}
```

Why

Prevents:

- parsing failures
 - hallucinated formats
 - inconsistent responses
-

14. Context Management Architecture

Problem

Large repositories exceed model context windows.

Solution

Use retrieval pipelines.

```
Repository
↓
Indexer
↓
Chunking
↓
Ranking
↓
Top Results
↓
Model
```

15. Context Allocation Strategy

Planner

Receives:

```
Small Context
```

Approximate content:

- goal
- plan
- memory summary

Reasoner

Receives:

```
Large Technical Context
```

Approximate content:

- files
- logs
- evidence

- graphs
-

Verifier

Receives:

Focused Review Context

Approximate content:

- findings
 - patches
 - test results
-

16. Memory Integration

Planner Memory

Stores:

Goals
Strategies
Plans

Reasoner Memory

Stores:

Architecture
Dependencies
Root Causes

Verifier Memory

Stores:

Failed Fixes
Regression Patterns
Review History

17. Long-Horizon Execution Support

Execution Cycle

```
Goal
↓
Plan
↓
Tool Use
↓
Observe
↓
Analyze
↓
Verify
↓
Continue
```

This cycle may repeat dozens or hundreds of times.

Checkpoint Integration

Models should never depend on conversation history alone.

State is persisted through:

```
Task State
Memory
Checkpoints
Events
```

18. Tool-Aware LLM Architecture

Models do not execute tools directly.

Correct Flow

```
Model
↓
Tool Request
↓
Orchestrator
↓
Tool
↓
Result
↓
Model
```

Benefits

- Security
- Traceability
- Recovery
- Monitoring

19. Retry Strategy

Temporary Failure

Retry

Repeated Failure

Fallback Model

Persistent Failure

Escalate To Orchestrator

20. Fallback Architecture

Future architecture:

```
Primary Model
  ↓
Failure
  ↓
Fallback Model
```

Example:

```
Kimi
  ↓
Unavailable
  ↓
Gemini
```

21. Rate Limit Handling

The gateway must manage:

```
OpenRouter Limits
Model Limits
User Limits
Task Limits
```

Queue Strategy

```
Request
  ↓
Queue
  ↓
Execution
  ↓
Response
```

22. Observability

Every model interaction must be logged.

Logged Fields

```
Model
Prompt Version
Latency
Token Usage
Cost
Success Rate
```

23. Security Requirements

Never expose:

```
API Keys
Secrets
Credentials
```

to models.

Context Sanitization

Before model calls:

```
Remove Secrets
Remove Tokens
Remove Credentials
```

24. Performance Targets

Planner

Target:

< 10 seconds

Reasoner

Target:

< 60 seconds

for large analysis tasks.

Verifier

Target:

< 20 seconds

25. Antigravity IDE Integration

Planner Output

Visible in:

Execution Timeline
Plan Workspace

Reasoner Output

Visible in:

Source Code Workspace
Findings Workspace
Patch Workspace

Verifier Output

Visible in:

Verification Workspace
Timeline Workspace

26. Future Expansion

Future architecture may include:

Research Agent
Review Agent
Coordinator Agent
Memory Agent

while preserving:

Planner
Reasoner
Verifier

as the core workflow.

27. Final LLM Architecture

```
User Goal
  ↓
Planner
(Kimi K2.6)
  ↓
Execution Plan
  ↓
Tools
  ↓
Evidence
  ↓
Reasoner
(Qwen3 Coder 480B)
  ↓
Finding
  ↓
```

```
Patch
  ↓
Verifier
(Nemotron 3 Ultra)
  ↓
Approve / Reject
  ↓
Execution
  ↓
Testing
  ↓
Verification
  ↓
Final Report
```

28. Architecture Summary

Agent Phantom Recovery uses a specialized three-model architecture optimized for:

- Long-horizon execution
- Repository analysis
- Tool orchestration
- Root cause investigation
- Patch generation
- Verification workflows
- Autonomous engineering tasks

The system deliberately separates:

```
Planning
Reasoning
Verification
```

into independent cognitive layers, creating a more reliable and scalable architecture than a traditional single-model approach.

End of Document

Status: Approved

Version: 1.0

Document Type: Production LLM Integration Architecture